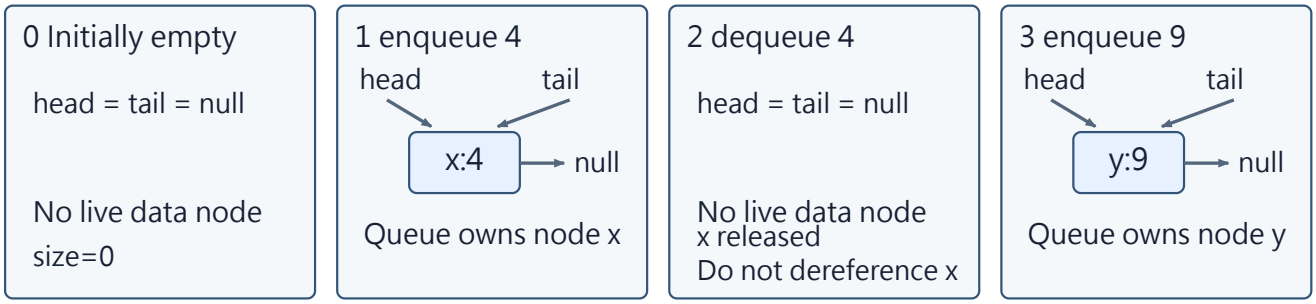
Chapter 4 worked solutions

DS04-01 • The last queue node

Initially head=tail=null. Enqueue 4 creates x and sets head=tail=x and x.next=null. Dequeue saves its value, removes x, and sets head=null. Because the queue is empty, also set tail=null before releasing x. Enqueue 9 creates y and sets head=tail=y. If tail still points to x after deletion, a subsequent write to tail.next accesses a released object. This violates ownership and lifetime, not merely displayed contents.

Last-node removal: clear head and tail before release



DS04-02 • Proving in-place reversal

Initially prev is empty and curr reaches $3 \to 8 \to 11$. After iteration 1, prev reaches 3 and curr $8 \to 11$. After iteration 2, prev reaches $8 \to 3$ and curr 11. After iteration 3, prev reaches $11 \to 8 \to 3$ and curr is empty. Saving curr.next before reversing the link preserves access to the remaining suffix. Both segments are acyclic, disjoint, and together contain exactly the original nodes. At termination set head=prev and tail=the old head. For an empty list both remain null. For a singleton they point to the same node.

DS04-03 • Polynomial normalization and addition

P repeats exponent 4 and is not strictly decreasing. Combining terms gives $P=x^4+x$, while $Q=-x^4+5$. Their sum is $x+5$. The zero coefficient at degree four produces no node. Comparison-sorting p unordered input terms costs O(p log p), followed by O(p) combination. Only addition of two already canonical lists costs O(p+q). Unsorted-input normalization cannot be hidden in that merge bound.

Chapter 4 worked solutions (continued)

DS04-04 • Doubly linked deletion

Save p=x.prev and q=x.next, set p.next=q and q.prev=p, release x, and decrement size. All other links are unchanged. The modified neighbors satisfy p.next.prev=p and q.prev.next=q, restoring local consistency. The precondition is that x is a live data node owned by this container, is not a sentinel, and has not already been deleted. An interface accepting arbitrary external pointers needs ownership validation or a stronger handle contract. It cannot treat every pointer as valid.

This procedure fixes two nondata sentinels H and T. H.prev=null and T.next=null; an empty list has H.next=T, T.prev=H, and size=0. Every data node has nonnull prev/next, making p.next and q.prev valid. A null-ended representation instead needs explicit head/tail repair when erasing an endpoint and cannot directly use this procedure.

For one item H↔x↔T, p=H and q=T. Erasure restores H.next=T and T.prev=H and decreases size from 1 to 0. Never release H or T. The same sentinel links cover first/last data nodes in longer lists. Validate the live owned handle before entering pointer operations.

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study